Hardware-Aware Speculative KV-Cache Paging and Epistemic Attention Eviction for High-Throughput Multi-Turn Agentic LLM Serving

Aniket Agrawal, *Google Cloud India Private Limited*

Abstract—The rapid proliferation of autonomous multi-turn agentic workflows has exacerbated the memory capacity bottleneck in Large Language Model (LLM) inference accelerators. In modern agentic sequences spanning 32k to 128k+ tokens, Key-Value (KV) cache memory footprints expand to 8.59 GB (8.00 GiB) per active 64k-token agent stream. Even under multi-chip tensor-parallel weight and KV-cache sharding or 32k–64k context distributions, this memory demand precipitates an acute capacity cliff on accelerators such as the Google Cloud TPU v6e Trillium (32 GB HBM per chip), exhausting remaining HBM capacity after only 3 to 8 concurrent sequences. Existing virtualized inference engines (e.g., PagedAttention / vLLM) mitigate Out-Of-Memory exceptions via reactive demand paging to Host DRAM over PCIe/CXL. However, servicing a synchronous full-layer demand page fault (64×64 KiB $= 4$ MiB across $L = 64$ layers, or $1.87\,\mu$s per single-layer 64 KiB block at 35 GB/s effective PCIe Gen5 bandwidth plus driver wakeup latency) incurs severe pipeline stalls (0.12 ms to 0.50 ms per faulted step), causing matrix execution units to idle for 40% to 70% of execution time. Furthermore, classical page eviction policies (LRU/LFU) are attention-blind: they inadvertently evict initial prompt attention sinks while preserving ephemeral chain-of-thought scratchpad tokens.

To overcome these constraints, we present **Speculative Epistemic KV-Cache Paging (SE-Paging)** governed by the **Epistemic Least-Likely-Attended (ELLA)** eviction protocol. SE-Paging couples the agent's semantic execution graph and turn-smoothed epistemic branch entropy $\mathcal{H}_i(t)$ to physical memory management: (1) an Epistemic Working Set Estimator (EWSE) projects future attention mass $\mathcal{A}(P_k, t)$, permanently pinning attention sinks and retaining active/idle tool schemas and DAG-boosted history while aggressively decaying scratchpad pages; (2) an Asynchronous DMA Speculative Prefetcher (ADSP) with sealed-page write-through host mirroring overlaps Host-to-Device page transfers with active matrix computation, eliminating device stall cycles under bounded prefetch queue depth; and (3) ELLA establishes a bounded competitive ratio relative to offline Belady MIN under the competitive caching with predictions framework. We evaluate SE-Paging in a downscaled $1:32$ trace-driven CPU-based physical tensor-buffer memory-hierarchy emulator (modeling a 2-layer 128 KiB slice per $B = 16$-token page, $N_{\text{dev}} = 96$ HBM pages [12.00 MiB], $N_{\text{host}} = 2{,}500$ DRAM pages [312.50 MiB], and 976 allocated pages [$15{,}616$ total tokens] across $K = 20$ streams) parameterized to scaled TPU v6e Trillium and PCIe Gen5 timing, complemented by a live multi-turn trace replay on Google Cloud Vertex AI (`gemini-2.5-flash`). Across $5{,}940$ logical page-lookup cycles ($11{,}880$ two-stage physical page transactions per protocol) spanning $K \in \{4, 8, 12, 16, 20\}$ concurrent agent streams under $10.17\times$ physical ($13.33\times$ unshared logical) working-set overflow, SE-Paging elevates cache hit rate at $K = 20$ from 0.20% (LRU) and 1.21% (LFU) to **28.23%**, eliminates 64.7% of memory pipeline stall overhead, and increases emulated wall-clock page-lookup throughput by 15.0% over LRU (+24.8% over LFU, or $2.60\times$ in pure simulated device throughput).

Index Terms—KV-Cache Memory Management, Hardware Accelerators, TPU v6e Trillium, Speculative DMA Prefetching, Multi-Agent LLM Serving, PagedAttention, Virtual Memory.

1 Introduction

Enterprise deployment of autonomous agent swarms built on foundation models (e.g., Gemini 2.5 Pro/Flash, Gemma 2, and Claude) represents a major workload shift in cloud computing. In frameworks such as the Google Agent Development Kit (ADK) and Vertex AI Agent Builder, agents engage in prolonged, multi-turn reasoning workflows involving hierarchical sub-goal decomposition, external tool invocations via OpenAPI schemas, and structured artifact generation [1], [2].

• *A. Agrawal is an AI Cloud Engineer at Google Cloud India Private Limited, Bangalore, India (e-mail: aniketagrawal@google.com).*

1.1 The KV-Cache HBM Capacity Cliff

Unlike traditional stateless NLP classification or single-turn dialogue, agentic sequences accumulate extensive conversational histories, tool schemas, and chain-of-thought scratchpads, routinely expanding context lengths from 32k to 128k+ tokens [3], [4].

During auto-regressive transformer decoding, Key and Value activation vectors for every past token must remain memory-resident to avoid redundant quadratic recomputation. The physical memory required scales linearly with context length S, transformer layers L, KV heads H_{kv}, and head dimension d_k:

$$\text{Memory}_{\text{KV}} = 2 \times L \times H_{\text{kv}} \times d_k \times S \times \text{bytes_per_elem} \quad (1)$$

For a representative enterprise model ($L = 64$, $H_{\text{kv}} = 8$, $d_k = 128$, `float16` with bytes_per_elem $= 2$), a sequence of $S = 64\text{k} = 65{,}536$ tokens requires $8{,}589{,}934{,}592$ bytes $\approx$ $\mathbf{8.59}$ GB decimal ($\mathbf{8.00}$ GiB binary) of dedicated High-Bandwidth Memory (HBM).

On modern cloud accelerators such as the Google Cloud TPU v6e Trillium (configured with 32 GB HBM in 256-chip 2D torus pods, following Google's custom TPU architectures [5]) or GPU nodes [6], this memory demand triggers an immediate *capacity cliff*: even under multi-chip tensor-parallel model-weight and KV-cache sharding (where 64-layer weights are partitioned across chips, leaving 16–24 GiB of non-weight HBM per chip) or 32k–64k context distributions with shared prefixes, a chip exhausts its remaining HBM pool after serving only 3 to 8 concurrent agent sequences, capping cluster concurrency.

1.2 The Demand Paging Pipeline Stall Dilemma

To prevent fatal Out-Of-Memory (OOM) aborts, modern serving runtimes (e.g., vLLM / PagedAttention [3] and SGLang [1]) partition the KV-cache into discrete non-contiguous physical pages ($B = 16$ tokens per block) and implement reactive *demand swapping* to Host DRAM over PCIe Gen5 or CXL, while multi-turn offloading systems such as CachedAttention [2] perform coarse-grained inter-turn session pre-loading across conversation boundaries.

However, intra-turn demand swapping is fundamentally reactive: when the accelerator's matrix execution units encounter an unmapped page during token generation, the decoding pipeline stalls. Over a PCIe Gen5 $\times 16$ interconnect (64 GB/s theoretical unidirectional bandwidth, ≈ 35 GB/s effective under multi-stream contention), transferring a single-layer 64 KiB page block ($B = 16, H_{\text{kv}} = 8, d_k = 128$) requires $\approx 1.87\,\mu$s of raw wire serialization; across all $L = 64$ transformer layers (4 MiB total per 16-token block) combined with PCIe DMA descriptor ring wakeup and kernel driver synchronization, a synchronous full-model page fault stalls token generation for 0.12 **ms to** 0.50 **ms** [6], [7]. Under high sequence concurrency, these synchronous demand stalls accumulate, causing matrix execution units to spend **40% to 70% of wall-clock time stalled waiting on memory**.

1.3 The Attention-Blind Eviction Failure

Furthermore, classical operating system page eviction policies (Least Recently Used [LRU], Least Frequently Used [LFU], FIFO) and existing token-pruning or single-step rehearsal heuristics are mismatched with multi-turn agentic transformer physics:

1) **Destruction of Attention Sinks and Tool Schemas:** Transformers concentrate massive attention mass on the initial 1 to 4 sink tokens [4] and active OpenAPI tool definitions. Under LRU, as an agent generates thousands of intermediate reasoning tokens, initial prompt tokens and tool schemas are aged out to Host RAM. While static window methods like StreamingLLM [4] drop middle tokens permanently, Heavy-Hitter Oracle (H2O) [8] tracks single-turn cumulative attention scores, Quest [9] estimates page criticality from query-key bounding boxes within the current layer, and InfiniGen [7] performs single-step layer-adjacent rehearsal prefetching (using skewed query-key projections in layer $l-1$ to prefetch layer l), none of these approaches satisfy multi-page PCIe lookahead horizons ($\tau_{\text{lookahead}}$ steps ahead) during abrupt multi-turn agentic tool-call DAG transitions where an entire dormant tool schema or prior history segment suddenly regains 100% attention relevance. Similarly, unlike purely static agentic workflow prefetchers (KVFlow [10]) or coarse session-level Time-to-Live (TTL) schedulers (PBKV [11], Continuum [12]) that ignore live generation uncertainty, SE-Paging couples workflow DAG lookahead with turn-smoothed epistemic Shannon entropy $\mathcal{H}_i(t)$.
2) **Scratchpad Memory Pollution:** Intermediate chain-of-thought scratchpad tokens are transient: once an agent concludes a reasoning sub-step, its scratchpad tokens are rarely re-attended. Classical LRU retains these newly written scratchpad pages in fast HBM, displacing reusable history and tool schemas.

1.4 Contributions

In this paper, we propose **Hardware-Aware Speculative KV-Cache Paging (SE-Paging)** governed by the **Epistemic Least-Likely-Attended (ELLA)** eviction protocol. Our key contributions are:

1) **Epistemic Working Set Estimator (EWSE):** A formal priority formulation that projects future page attention mass $\mathcal{A}(P_k, t)$ from agent semantic segment tags, tool-call and history DAG lookahead, and turn-smoothed epistemic Shannon entropy $\mathcal{H}_i(t)$.
2) **Asynchronous DMA Speculative Prefetcher (ADSP):** A non-blocking prefetcher with sealed-page write-through host mirroring that dispatches Host-to-Device transfers ahead of execution steps, overlapping PCIe/CXL transfer latency with matrix compute.
3) **Theoretical Performance Guarantees:**
   - **Theorem 1 (Queued Zero-Stall Pipeline Overlap):** Establishes the step-indexed multi-page lookahead condition $\tau_{\text{lookahead}} \geq \left\lceil \frac{M_{\text{pre}}(n) T_{\text{DMA}}(P_k)}{(1-\rho_{\text{bus}}) T_{\text{compute}}} \right\rceil$ under which device stall cycles are identically zero.
   - **Theorem 2 (Attention Mass Preservation):** Proves that ELLA scratchpad eviction beyond the pinned active window ($\delta = 64$ tokens) analytically bounds attention mass loss by $\epsilon_{\text{attn}} \leq C\delta^{1-\gamma}\left(\frac{1}{\delta} + \frac{1}{\gamma-1}\right) \leq \frac{C_{\text{scratch}}}{(\gamma-1)\delta^{\gamma-1}}$ (≤ 0.0261 exact analytically at $\delta = 64$, ≤ 0.0105 at $\delta = 256$, with empirical trace loss $6.4 \times 10^{-4} < 10^{-3}$, or $> 99.9\%$ preservation).
   - **Theorem 3 (Bounded Competitive Ratio):** Uses the competitive caching with predictions framework [13] on acyclic agentic execution graphs to prove that ELLA with inverse forward DAG distance ordering achieves a com-

petitive ratio within $\mathcal{C}_0 + \mathcal{O}\left(\frac{\eta_0}{\mathrm{OPT}} + \mathcal{H}_{\max}\right)$ of offline Belady MIN [14].

4) **Downscaled Physical Tensor-Buffer Emulator & Live Vertex AI Trace Replay:** We evaluate SE-Paging in a downscaled $1:32$ CPU-based physical tensor-buffer emulator (modeling a 2-layer 128 KiB slice per page, $N_{\text{dev}} = 96$ HBM pages [12.00 MiB], $N_{\text{host}} = 2{,}500$ DRAM pages [312.50 MiB], and 976 physical pages [15,616 total tokens] across $K = 20$ streams) across 5,940 logical page-lookup cycles (11,880 two-stage physical page transactions per protocol), including ablations against `DEMAND_ELLA`, `STATIC_DAG_ELLA` ($\mathcal{H}_i(t) \equiv 0$, KVFlow-style [10]), `PBKV/Continuum` [11], [12], `StreamingLLM` [4], `H2O` [8], `CachedAttention` [2], and `InfiniGen` [7], and validate on a live multi-turn Google Cloud Vertex AI (`gemini-2.5-flash`) agent trace replay.

2 Related Work

- **Virtualized KV-Cache and Offloading Systems:** PagedAttention / vLLM [3] introduced OS-inspired block-level KV-cache virtualization ($B = 16$ tokens) with reactive demand swapping, while SGLang [1] introduced RadixAttention for prefix tree reuse. FlexGen [6] formulated static linear-programming offload schedules across GPU, CPU DRAM, and disk for offline batch throughput, and CachedAttention [2] overlapped inter-turn session restoration across multi-turn conversation boundaries. However, none of these systems prevent intra-turn page-fault stalls when an active agent dynamically switches between tool schemas and history blocks across sub-steps.
- **Token Pruning and Intra-Layer Sparse Attention:** StreamingLLM [4] retains initial attention sinks plus a static sliding window while permanently discarding middle tokens. H2O [8] evicts tokens with low cumulative attention scores, Quest [9] selects top-k pages per layer using query-key bounding boxes without Host-to-Device DMA lookahead, and InfiniGen [7] prefetches KV entries from layer $l-1$ to layer l via skewed projections. While effective for single-step layer overlap, single-layer lookahead ($\tau = 1$ layer) is insufficient to hide multi-page PCIe Gen5 transfer latency across multi-turn agentic state transitions.
- **Agent-Aware KV-Cache Management:** Recent agentic serving schedulers including KVFlow [10], PBKV [11], and Continuum [12] exploit static agentic workflow graphs or coarse session-level Time-to-Live (TTL) estimates to pre-warm or pin agent prefixes. However, static DAG distances and session TTLs are deterministic: when an agent encounters high epistemic branching uncertainty during reasoning or tool error recovery, static schedules either evict active scratchpad context prematurely or fail to boost history segments needed for verification. SE-Paging bridges this gap by combining fine-grained categorical page tagging ($\mathcal{P}_{\text{anchor}}, \mathcal{P}_{\text{tool}}, \mathcal{P}_{\text{active}}, \mathcal{P}_{\text{hist}}, \mathcal{P}_{\text{scratch}}$) and forward DAG distance $\hat{d}_{\text{DAG}}(P_k)^{-1}$ with live turn-smoothed epistemic branch entropy $\mathcal{H}_i(t)$ and sealed-page write-through DMA mirroring.

3 System Architecture and Memory Hierarchy

3.1 Heterogeneous Memory Hierarchy Model

We model an AI inference accelerator node (e.g., Google Cloud TPU v6e Trillium programmed via JAX/Pallas kernels [15]; https://jax.readthedocs.io/en/latest/pallas/) comprising two memory tiers:

- **Tier 0 (Device HBM):** High-bandwidth, capacity-constrained memory directly attached to Matrix Execution Units (MXUs) ($\text{BW}_{\text{HBM}} \geq 1.6$ TB/s, capacity $C_{\text{HBM}} \in \{32, 64\}$ GB).
- **Tier 1 (Host DRAM / CXL):** High-capacity system memory connected via PCIe Gen5 / CXL ($\text{BW}_{\text{bus}} \approx$ 32–64 GB/s, capacity $C_{\text{Host}} \geq 512$ GB).

KV-cache is managed in fixed-size physical page blocks P_k containing $B = 16$ tokens. For a single transformer layer ($H_{\text{kv}} = 8, d_k = 128$, `float16`), each page block occupies:

$$\begin{aligned}\text{Size}_{\text{1-layer}}(P_k) &= B \times 2 \times H_{\text{kv}} \times d_k \times 2\ \text{B} \\ &= 65{,}536\ \text{B}\ (64\ \text{KiB})\end{aligned} \tag{2}$$

In our multi-layer block allocator (Section 6.1), pairs of adjacent transformer layers are co-allocated into fused 128 KiB (131,072 byte) physical transfer units to amortize PCIe DMA descriptor ring overhead while permitting layer-pipelined DMA interleaving and sub-stack preemption across concurrent streams without blocking on a monolithic 4 MiB transfer. In our scaled $1:32$ slice emulator (modeling a 2-layer 128 KiB slice out of $L = 64$ layers), both per-slice DMA transfer latency $T_{\text{DMA}}(P_k) = 3.74\,\mu$s and per-slice ideal matrix compute time $T^{\text{ideal}}_{\text{compute}} = 5.0\,\mu$s scale uniformly by $1:32$ relative to a full 64-layer step ($T^{\text{64L}}_{\text{DMA}} \approx 119.7\,\mu$s, $T^{\text{64L}}_{\text{compute}} \approx 160\,\mu$s), preserving the exact dimensionless overlap ratio $T_{\text{DMA}}(P_k)/T_{\text{compute}}$, page-replacement hit rates, and working-set overflow ratios.

3.2 Categorical Page Classification and Boundary Resolution

As tokens are ingested or generated by the orchestrator (e.g., Google ADK or SGLang [1]), lightweight structural delimiters (`<|system|>`, `<|tool_schema|>`, `<|history|>`, `<|thought|>`) tag each token span in $\mathcal{O}(1)$ time with a persistent underlying structural class ($\mathcal{P}_{\text{anchor}}, \mathcal{P}_{\text{tool}}, \mathcal{P}_{\text{hist}}, \mathcal{P}_{\text{scratch}}$), while dynamically maintaining an active sliding window ($\mathcal{P}_{\text{active}}$):

1) **System Anchor ($\mathcal{P}_{\text{anchor}}$):** System prompts and invariant instructions acting as persistent attention sinks [4].
2) **Tool Schema ($\mathcal{P}_{\text{tool}}$):** OpenAPI function declarations and JSON schema contracts.
3) **Active Generation Window ($\mathcal{P}_{\text{active}}$):** A temporary sliding-window boost applied to the most recent

$W_{\text{local}} = 64$ tokens (4 pages of $B = 16$ tokens) at the active decoding frontier.

4) **Conversation History ($\mathcal{P}_{\text{hist}}$):** Verified multi-turn user queries and committed tool outputs.
5) **Intermediate Scratchpad ($\mathcal{P}_{\text{scratch}}$):** Ephemeral chain-of-thought reasoning tokens from completed turns.

Dominant-Class Boundary and Non-Demoting Sliding-Window Rule: Each page permanently stores its underlying delimiter class $c(P_k) \in \{\mathcal{P}_{\text{anchor}}, \mathcal{P}_{\text{tool}}, \mathcal{P}_{\text{hist}}, \mathcal{P}_{\text{scratch}}\}$ resolved under the dominance hierarchy $\mathcal{P}_{\text{anchor}} \succ \mathcal{P}_{\text{tool}} \succ \mathcal{P}_{\text{hist}} \succ \mathcal{P}_{\text{scratch}}$ whenever a $B = 16$-token block straddles two delimiter spans. For any page P_k intersecting the most recent $W_{\text{local}} = 64$ tokens, its effective priority is evaluated as $\tilde{\mathcal{A}}_i(P_k, t) = \max(\mathcal{A}_{\text{class},i}(P_k, t), \mathcal{A}_{\text{active},i}(P_k, t))$, ensuring that an active tool schema ($\mathbb{I}_{\text{tool},i} = 1$, score 5×10^8) or anchor (10^9) inside W_{local} is never demoted to 10^7, while history and scratchpad pages inside W_{local} are elevated to $\mathcal{A}_{\text{active},i}(P_k, t) = 10^7 \exp(-\kappa \Delta t_{\text{win}})$ and automatically revert to $\mathcal{A}_{\text{class},i}(P_k, t)$ once the decoding frontier advances past 64 tokens.

4 The Speculative Epistemic Paging Protocol

4.1 Epistemic Attention Scoring Formulation

Definition 1 (Projected Attention Score)**.** *For a KV-cache page P_k evaluated for agent sequence S_i at logical agentic turn (or sub-goal step) t, its per-stream class score $\mathcal{A}_{class,i}(P_k, t)$ and sliding-window score $\mathcal{A}_{active,i}(P_k, t)$ under ELLA are defined as:*

$$\mathcal{A}_{\text{class},i}(P_k,t) \triangleq \begin{cases} 10^9, & c(P_k) = \mathcal{P}_{anchor} \\ 5\times 10^8\, \mathbb{I}_{tool,i}(P_k,t) + (1-\mathbb{I}_{tool,i}(P_k,t))(5\times 10^4 e^{-\lambda_{tool}\Delta t} + C_{floor}), & c(P_k) = \mathcal{P}_{tool} \\ \max(C_{hist}, 10^4 e^{-\lambda\Delta t}(1-\mathcal{H}_i(t)+\epsilon)) + B_{DAG}\mathbb{I}_{DAG,i}(P_k,t) + \eta_{DAG}\hat{d}_{DAG}(P_k)^{-1}, & c(P_k) = \mathcal{P}_{hist} \\ 10^2 \exp(-\mu\Delta t)\,\mathcal{H}_i(t) + \eta_{DAG}\hat{d}_{DAG}(P_k)^{-1}, & c(P_k) = \mathcal{P}_{scratch} \end{cases} \tag{3}$$

with effective per-stream score $\tilde{\mathcal{A}}_i(P_k, t) = \max(\mathcal{A}_{class,i}(P_k, t), 10^7 \exp(-\kappa \Delta t_{win}) \mathbb{I}[P_k \in \mathcal{P}_{active,i}])$, and global shared-page priority across all referencing streams $\mathcal{O}(P_k)$:

$$\mathcal{A}(P_k, t) \triangleq \max_{S_i \in \mathcal{O}(P_k)} \tilde{\mathcal{A}}_i(P_k, t) \tag{4}$$

where $\mathbb{I}_{tool,i}(P_k, t) \in \{0, 1\}$ and $\mathbb{I}_{DAG,i}(P_k, t) = \mathbb{I}[\hat{d}_{DAG}(P_k) \leq d_{look}] \in \{0, 1\}$ equal 1 when the orchestrator's workflow DAG or grammar-constrained router predicts a reference to tool schema or history page P_k within lookahead depth $d_{look} = 2$ steps (and 0 otherwise); $C_{floor} = 200$ and $C_{hist} = 120$ are the asymptotic tool-schema and verified-history retention floors; $B_{DAG} = 350.0$ is the history lookahead boost (raising DAG-predicted history pages to $\geq C_{hist} + B_{DAG} = 470 > C_{floor} = 200$ so ADSP can preempt decayed idle tool schemas when resuming a prior conversation branch); $\Delta t = t - t_{last_access}(P_k) \geq 0$ is the elapsed logical turn count since the semantic segment containing P_k was generated or referenced by the orchestrator (avoiding per-token attention-matrix materialization under fused Pallas FlashAttention-2 kernels [16], while $\Delta t_{win} \in \{0, 1, 2, 3\}$ indexes 16-token block age within $W_{local} = 64$); $\hat{d}_{DAG}(P_k)^{-1} \in [0, 1]$ is the inverse forward step distance to the next workflow node referencing P_k (0 if $\hat{d}_{DAG}(P_k) = \infty$, scaled by $\eta_{DAG} = 10$); and decay hyperparameters are calibrated via grid search on held-out traces to $\kappa = 0.05$, $\lambda = 0.08$, $\lambda_{tool} = 0.04$, $\mu = 0.45$, and $\epsilon = 0.10$ ($\mu \gg \lambda > \kappa > 0$). Ties are broken deterministically by $\hat{d}_{DAG}(P_k)^{-1}$, then LRU timestamp, and finally physical page index k.

Action-Delimiter Epistemic Entropy $\mathcal{H}_i(t)$: To prevent deterministic JSON punctuation and syntax tokens from collapsing entropy toward zero, the orchestrator evaluates Shannon entropy exclusively at structural branch and action delimiters $m \in \mathcal{T}_{\text{branch}}$ (e.g., <|thought|>, <|tool_call|>, and tool-name selection tokens) over the candidate action-token subset $\mathcal{V}_{\text{act}}$ ($|\mathcal{V}_{\text{act}}| = 64$), smoothed via an exponential moving average with factor $\beta = 0.3$:

$$\mathcal{H}_i(t) = (1-\beta)\mathcal{H}_i(t-1) + \beta\left(-\frac{1}{\ln|\mathcal{V}_{\text{act}}|}\sum_{v\in\mathcal{V}_{\text{act}}} p_{i,m}(v)\ln p_{i,m}(v)\right) \in [0,1] \tag{5}$$

with $0 \ln 0 \triangleq 0$. During high-uncertainty exploration ($\mathcal{H}_i(t) \to 1$), Eq. (3) weights recent scratchpad pages upward while annealing history priority toward C_{hist}; once branching resolves ($\mathcal{H}_i(t) \to 0$), scratchpad priority collapses to ≤ 10 ($\eta_{\text{DAG}} \hat{d}_{\text{DAG}}^{-1}$). Furthermore, because idle tool schemas retain $\geq C_{\text{floor}} = 200$ and history retains $\geq C_{\text{hist}} = 120 > 110$ (the maximum possible scratchpad score $10^2 + \eta_{\text{DAG}} = 110$ at $\Delta t = 0, \mathcal{H}_i(t) = 1$), neither dormant tool schemas nor verified conversation history ever decay below scratchpad priority over arbitrarily long horizons ($\Delta t \to \infty$).

4.2 Epistemic Least-Likely-Attended (ELLA) Eviction Invariant

Invariant 1 (ELLA Evictable Victim Selection)**.** *Let $Resident_{evictable}(HBM) \triangleq Resident(HBM) \setminus (\mathcal{P}_{pinned_prefetch} \cup \mathcal{P}_{active_unsealed})$ denote the subset of HBM-resident pages excluding in-flight/admitted prefetch-pinned pages ($\leq \tau_{lookahead} M_{pre}^{\max} = 8$ pages) and currently unsealed active tail pages ($\leq K$ pages). When allocating or prefetching a page requires a free frame in Device HBM, the controller evicts:*

$$P_{victim} = \arg \min_{P_j \in Resident_{evictable}(HBM)} \mathcal{A}(P_j, t) \tag{6}$$

If $Resident_{evictable}(HBM) = \emptyset$ under extreme concurrency, ADSP suppresses speculative prefetch admission and releases the lowest-scoring prefetch-pinned page prior to evicting any $\mathcal{P}_{anchor}$ page (10^9).

4.3 Asynchronous DMA Speculative Prefetcher and Write-Through Mirroring

To avoid blocking Device-to-Host writebacks during eviction, SE-Paging implements **Asynchronous Sealed-Page Write-Through Mirroring**: while the active tail page in $\mathcal{P}_{\text{active}}$ ($B = 16$ tokens) is populated token-by-token across 16 decoding steps, it remains pinned in HBM; once all 16 token slots are filled and the page is sealed, a single asynchronous background DMA copy mirrors the full 128 KiB (4 MiB full-stack) clean page to its pre-allocated Host DRAM slot, avoiding $16\times$ partial-page PCIe write amplification. Because KV-cache tokens are append-only in causal transformer decoding, sealed pages are never mutated in-place; if a sealed page is evicted before its background write-through completes, the controller awaits

completion of the in-flight write-through descriptor before reusing the HBM frame. Prior to logical step n, ADSP aggregates candidate non-resident pages across all active streams (breaking ties by round-robin stream index i and inverse DAG distance $\hat{d}_{\text{DAG}}(P_k)^{-1}$), selects up to $M_{\text{pre}}(n) \leq 4$ candidates $P_{\text{cand},r}$ in descending order of $\mathcal{A}(P_{\text{cand},r}, n)$ against evictable HBM victims $P_{\text{victim},r}$ in ascending order, and dispatches asynchronous Host-to-Device DMA transfers whenever $\mathcal{A}(P_{\text{cand},r}, n) > \mathcal{A}(P_{\text{victim},r}, n)$, pinning admitted pages in $\mathcal{P}_{\text{pinned_prefetch}}$ until step n completes.

5 Formal Theoretical Analysis

5.1 Theorem 1: Queued Zero-Stall Pipeline Overlap Invariant

Theorem 1 (Zero-Stall Overlap Under Queued DMA)**.** *Let $T_{compute}$ be the per-step execution latency of matrix execution units, let $T_{DMA}(P_k) = Size(P_k)/BW_{bus}$ be the raw Host-to-Device transfer time of page P_k, let $M_{pre}(n) = |\mathcal{W}_{spec}(n)|$ be the batch prefetch count dispatched for step n (and $M_{pre}^{\max} = \max_n M_{pre}(n)$ its global cap), and let $\rho_{bus} \in [0, 1)$ denote background interconnect utilization (incorporating concurrent stream prefetches and full-duplex write-through mirroring traffic). If the speculative lookahead horizon (in execution steps) satisfies:*

$$\tau_{lookahead} \geq \left\lceil \frac{M_{pre}(n) \cdot T_{DMA}(P_k)}{(1 - \rho_{bus}) \cdot T_{compute}} \right\rceil \tag{7}$$

and $M_{pre}(n)$ clean sealed victim frames in $Resident_{evictable}(HBM)$ (disjoint from active working sets over $[t_{dispatch}, t_n)$) are available in HBM, then step n experiences identically zero memory stall cycles ($\Delta_{stall}(n) \equiv 0$).

Proof. Let step n require the non-resident prefetched working-set subset $\mathcal{W}_{\text{spec}}(n)$ of size $M_{\text{pre}}(n)$ at wall-clock time t_n. Under ADSP, the batch DMA request is dispatched at $t_{\text{dispatch}} = t_n - \tau_{\text{lookahead}} \cdot T_{\text{compute}}$. Under available bus bandwidth $(1 - \rho_{\text{bus}})\text{BW}_{\text{bus}}$ and write-through clean victim reclamation, the serialized completion time of all $M_{\text{pre}}(n)$ pages in the DMA queue is:

$$t_{\text{completion}} = t_{\text{dispatch}} + \frac{M_{\text{pre}}(n) \cdot T_{\text{DMA}}(P_k)}{1 - \rho_{\text{bus}}} \tag{8}$$

Substituting Eq. (7) yields $\tau_{\text{lookahead}} \cdot T_{\text{compute}} \geq \frac{M_{\text{pre}}(n) T_{\text{DMA}}(P_k)}{1-\rho_{\text{bus}}}$, which implies $t_{\text{completion}} \leq t_n$. Thus all $M_{\text{pre}}(n)$ pages are mapped in HBM prior to t_n, and $\Delta_{\text{stall}}(n) = \max(0, t_{\text{completion}} - t_n) = 0$. When extreme overcommit ($K = 20$, e.g., $M_{\text{pre}}(n) = 4$ where $\lceil 4 \times 3.74/(0.82 \times 5.0) \rceil = 4 > \tau_{\text{lookahead}} = 2$) causes $M_{\text{pre}}(n)$ to exceed the lookahead window, wall-clock stall time is reduced strictly by the overlapped lookahead duration $\tau_{\text{lookahead}} T_{\text{compute}}$ (corresponding to $\tau_{\text{lookahead}}(1-\rho_{\text{bus}})T_{\text{compute}}$ of uncontended DMA transfer work). □

5.2 Theorem 2: Attention Mass Bounded Approximation

Theorem 2 (Attention Mass Preservation)**.** *Under Rotary Position Embeddings (RoPE [17]) where post-softmax normalized attention weights ($\sum_{n=1}^{m} \alpha_{m,n} = 1$) on completed scratchpad tokens decay with relative token distance $|m - n| \geq \delta$ as $\alpha_{m,n} \leq C|m - n|^{-\gamma}$ ($\gamma > 1$), evicting scratchpad pages outside the pinned active window at token distance $\geq \delta = W_{local}$ under ELLA preserves at least $1 - \epsilon_{attn}$ of the full-context attention mass, where:*

$$\epsilon_{attn} \leq C\delta^{1-\gamma}\left(\frac{1}{\delta} + \frac{1}{\gamma - 1}\right) \leq \frac{C_{scratch}}{(\gamma - 1)\,\delta^{\gamma-1}}, \quad C_{scratch} \triangleq C\gamma \tag{9}$$

Proof. ELLA permanently retains all 4 $\mathcal{P}_{\text{anchor}}$ pages (64 sink tokens) and pins the 4 $\mathcal{P}_{\text{active}}$ pages ($W_{\text{local}} = 64$ tokens, $|m - n| < \delta = 64$) in HBM. Because the attention sink and local window tokens alone contribute at least $Z_{\text{pinned}} = \sum_{j \in \mathcal{P}_{\text{anchor}} \cup \mathcal{P}_{\text{active}}} \exp(q_m^\top k_j / \sqrt{d_k})$ to the softmax partition denominator $Z_m \geq Z_{\text{pinned}}$, the normalized per-token weight on any distant scratchpad token $|m - n| \geq \delta$ satisfies $\alpha_{m,n} \leq C|m - n|^{-\gamma}$ with calibrated scale constant $C = 0.25$ and decay exponent $\gamma = 1.65$. Bounding the discrete token sum over evicted $\mathcal{P}_{\text{scratch}}$ tokens at distance $x = |m - n| \geq \delta \geq 1$ up to history length $S_{\text{history}} \leq S$ via the integral test and relaxing $S_{\text{history}} \to \infty$:

$$\begin{aligned}\sum_{j \in \mathcal{P}_{\text{evicted}}} \alpha_{m,j} &\leq C\delta^{-\gamma} + \int_{\delta}^{S_{\text{history}}} Cx^{-\gamma}\,dx \\ &\leq C\delta^{-\gamma} + \int_{\delta}^{\infty} Cx^{-\gamma}\,dx \\ &= C\delta^{1-\gamma}\left(\frac{1}{\delta} + \frac{1}{\gamma - 1}\right) \leq \frac{C\gamma}{(\gamma - 1)\delta^{\gamma-1}}\end{aligned} \tag{10}$$

since $\frac{1}{\delta} \leq 1$ for $\delta \geq 1$. At the guaranteed active-window boundary $\delta = W_{\text{local}} = 64$ tokens ($\gamma = 1.65, C = 0.25$), the exact analytical bound is $\epsilon_{\text{attn}} \leq 0.0261$ (and $\epsilon_{\text{attn}} \leq 0.0105$ when recent history spans $\delta = 256$ tokens), while the empirically measured attention mass on evicted scratchpad pages across all $K = 20$ traces is $\epsilon_{\text{attn}}^{\text{emp}} = 6.4 \times 10^{-4} < 10^{-3}$ (99.93% retained pre-fault attention mass). □

5.3 Theorem 3: Bounded Competitive Ratio Under Epistemic Predictions

Theorem 3 (Competitive Ratio vs. Belady MIN)**.** *On an acyclic agentic execution DAG where ELLA orders pages via Eq. (3) using inverse forward DAG distance $\hat{d}_{DAG}(P_k)^{-1}$, under trace regularity (or with baseline per-phase ℓ_1 rank-inversion error η_0 from finite-decay crossovers and turn-smoothed branch entropy $\mathcal{H}_i(t) \leq \mathcal{H}_{\max}$), ELLA achieves a competitive ratio:*

$$\mathcal{C}_{ELLA} \leq \mathcal{C}_0 + \mathcal{O}\left(\frac{\eta_0}{OPT} + \mathcal{H}_{\max}\right) \tag{11}$$

relative to offline Belady MIN [14] ($\mathcal{C}_0 = 1$ under trace-regular greedy ordering, or $\mathcal{C}_0 = 2$ when paired with Predictive Marker phases [13]).

Proof. Belady MIN [14] evicts the resident page whose next access time $t_{\text{next}}(P_k)$ is furthest in the future. In the deterministic limit $\mathcal{H}_i(t) \to 0$, completed unreferenced scratchpad pages ($t_{\text{next}} = \infty$) have score 0, anchors ($t_{\text{next}} = t + 1$) have score 10^9, and pending tools ($t_{\text{next}} \leq t + 2$) have score 5×10^8. Under trace regularity (where future reuse respects the semantic tier hierarchy and intra-tier reuse follows $\hat{d}_{\text{DAG}}(P_k)^{-1}$), ELLA's eviction ordering is order-isomorphic to $t_{\text{next}}(P_k)$ ($\eta_0 = 0, \mathcal{C}_0 = 1$); when finite-decay crossovers between idle $\mathcal{P}_{\text{tool}}$ and $\mathcal{P}_{\text{hist}}$ occur, they incur a bounded

baseline ℓ_1 rank-inversion error η_0. Under stochastic branching with turn-smoothed branch entropy $\mathcal{H}_i(t) \leq \mathcal{H}_{\max}$, the branch misprediction probability at each DAG decision node is upper-bounded by $\mathcal{O}(\mathcal{H}_{\max})$. Summing the induced rank inversions across all $\Theta(\text{OPT})$ cache phases yields total expected prediction error $\mathbb{E}[\eta]/\text{OPT} \leq \mathcal{O}(\eta_0/\text{OPT} + \mathcal{H}_{\max})$, establishing Eq. (11) via the competitive caching with predictions theorem of Lykouris and Vassilvitskii [13]. □

6 Empirical Evaluation and Benchmark Results

6.1 Experimental Methodology and Downscaled 1:32 Physical Tensor Emulator

Following established LLM serving simulation methodologies such as Vidur [18], we evaluate multi-tier paging deterministically without nondeterministic network jitter via a **downscaled** $1:32$ **trace-driven CPU-based physical tensor-buffer memory-hierarchy emulator** in Python/NumPy on an Apple Silicon workstation (8 CPU cores, APFS storage), parameterized to scaled TPU v6e Trillium and PCIe Gen5 timing, complemented by a live multi-turn trace replay on Google Cloud Vertex AI (`gemini-2.5-flash`):

- **Physical Tensor Page Pools & Timing Constants:** Modeling a 2-layer fused KV-cache slice (128 KiB per $B = 16$-token page, scaling $1:32$ relative to a full $L = 64$-layer 4 MiB stack), we allocate two contiguous `numpy.float16` tensor arenas representing Tier 0 Device HBM ($N_{\text{dev}} = 96$ pages $\times$ 128 KiB/page $=$ 12.00 MiB, or 12.58 MB decimal) and Tier 1 Host DRAM ($N_{\text{host}} = 2{,}500$ pages $\times$ 128 KiB/page $=$ 312.50 MiB, or 327.68 MB decimal). Every page fault and prefetch executes a physical 128 KiB `float16` memory copy between Host and Device arenas paired with calibrated $1:32$-scaled timing constants: per-access ideal device compute $T^{\text{ideal}}_{\text{compute}} = 5.0\,\mu$s, raw PCIe Gen5 ($\text{BW}_{\text{bus}} = 35$ GB/s) slice transfer $T_{\text{DMA}}(P_k) = 3.74\,\mu$s plus driver wakeup ($95.0\,\mu$s synchronous demand-fault penalty vs. $31.2\,\mu$s partially overlapped prefetch penalty when $M_{\text{pre}}(n)$ exceeds $\tau_{\text{lookahead}}$), background bus utilization $\rho_{\text{bus}} = 0.18$ (including Device-to-Host write-through mirroring), lookahead horizon $\tau_{\text{lookahead}} = 2$ steps ($M_{\text{pre}}(n) \leq 4$ pages/step globally), and measured host Python arena-copy overhead $T_{\text{host_arena}} \approx 1.4\,\mu$s.
- **Shared Warm Prefix, Trace Parameterization & Overflow Ratios:** Across $K \in \{4, 8, 12, 16, 20\}$ concurrent agent streams, all streams share a pre-warmed 16-page swarm prefix ($4\ \mathcal{P}_{\text{anchor}} + 12\ \mathcal{P}^{\text{shared}}_{\text{tool}}$, 256 shared tokens resident in HBM at $t = 0$) and allocate 48 private pages per stream ($4\ \mathcal{P}_{\text{active}}, 16\ \mathcal{P}_{\text{hist}}, 4\ \mathcal{P}^{\text{priv}}_{\text{tool}}, 24\ \mathcal{P}_{\text{scratch}} = 768$ private tokens, or 64 logical pages $= 1{,}024$ logical tokens per stream), totaling $16 + 48K$ distinct physical pages (976 physical pages $= 15{,}616$ total tokens at $K = 20$, representing a $976/96 = 10.17\times$ physical HBM overflow, or $20 \times 64/96 = 13.33\times$ unshared logical working-set overflow, which matches the exact working-set-to-HBM ratio of 20×32k-token streams on a 32 GB TPU v6e chip). Within each turn (8–12 steps/turn, $\mathcal{H}_i(t) \sim \text{Beta}(2, 5)$, tool transitions $\mathbb{I}_{\text{tool},i}$ every 2 turns), each stream re-references a hot kernel of ≈ 4.5–4.7 private pages alongside streaming one-pass scratchpad allocations. Table 1 reports canonical seed-0 hit counts alongside 5-seed Mean $\pm$ SD throughput; Table 2 reports 5-seed Mean $\pm$ SD across all 9 policies.
- **Transaction Accounting ($N_{\text{tx}} = 11{,}880$ Physical Transactions per Protocol):** Each stream executes 99 logical page-lookup cycles ($396, 792, 1{,}188, 1{,}584$, and $1{,}980$ accesses for $K = 4, 8, 12, 16, 20$, summing to $5{,}940$ logical accesses per protocol). Because each cycle performs 2 physical arena operations (a Tier-0 lookup/read plus a tensor state/metadata update, including a write-through check under SE-Paging), each protocol executes $2 \times 5{,}940 = 11{,}880$ physical page transactions.
- **Metrics & Baseline Adaptation:** `Wall Throughput (Ops/s)` is $\text{Accesses}/T_{\text{wall}}$ where $T_{\text{wall}} = T^{\text{ideal}}_{\text{compute}} + \sum T_{\text{stall}} + T_{\text{host_arena}}$ (guaranteeing `Wall Throughput` $<$ `Sim. Dev. Ops/s` for every row since $T_{\text{host_arena}} > 0$), while `Sim. Dev. Ops/s` $= \text{Accesses}/(T^{\text{ideal}}_{\text{compute}} + \sum T_{\text{stall}})$ isolates simulated accelerator throughput. `Stall Ratio (%)` is $\left(\sum T_{\text{stall}}/T^{\text{ideal}}_{\text{compute}}\right) \times 100\%$. `Attention Retained` in Table 2 measures the pre-fault HBM resident fraction $\sum_{j \in \text{Resident(HBM)}} \alpha_{m,j}$ of calibrated RoPE power-law attention weights ($\gamma = 1.65, C = 0.25$) satisfied immediately prior to synchronous demand-fault servicing. Single-tier pruning baselines (`StreamingLLM` [4], `H2O` [8]) are adapted into lossless two-tier $B = 16$ block-replacement policies by demoting evicted blocks to Tier 1 Host DRAM (faulted back on re-reference), and proactive/agentic baselines include `CachedAttention` [2], `InfiniGen` [7], `STATIC_DAG_ELLA` ($\mathcal{H}_i(t) \equiv 0$, KVFlow-style static DAG prefetching [10]), and `PBKV/Continuum` [11], [12] (session TTL eviction).

6.2 Live Multi-Turn Vertex AI (`gemini-2.5-flash`) Trace Replay

To validate SE-Paging on authentic LLM agent execution traces beyond synthetic $\text{Beta}(2, 5)$ entropy sampling, we executed a live multi-turn agentic workflow on Google Cloud Vertex AI (`project="aniket-personal"`, `model="gemini-2.5-flash"`) across 3 reasoning turns (System/Tool Ingestion $\rightarrow$ Cloud Spanner Diagnostic Tool Invocation $\rightarrow$ Remediation Verification, generating $1{,}828$ live tokens with mean API latency 2,811.3 ms/turn) and replayed the resulting delimiter-tagged token stream across $K = 20$ concurrent streams ($1{,}980$ page accesses, 110 hot pages $> N_{\text{dev}} = 96$). On the live `gemini-2.5-flash` trace replay, `DEMAND_LRU` and `DEMAND_LFU` suffered severe thrashing (0.20% and 1.21% hit rates; $1{,}954.5\%$ and $1{,}937.6\%$ Stall Ratios), whereas `SPECULATIVE_ELLA` achieved a **28.48% hit rate** (564 hits including 59 completed prefetches), reduced Stall Ratio by

TABLE 1
Physical Tensor-Arena Paging Telemetry: DEMAND_LRU vs. DEMAND_LFU vs. SPECULATIVE_ELLA ($N_{\text{dev}} = 96$ Pages, 5,940 Logical Accesses / 11,880 Physical Two-Stage Page Ops per Protocol, Canonical Seed-0 Hit Counts & 5-Seed Throughput Mean ± SD)

Protocol	Streams (K)	Page Accesses	Cache Hits‡	Hit Rate (%)	Prefetch Hits‡	Wall Throughput (Ops/s)	Sim. Dev. Ops/s§	Stall Ratio (%)†
DEMAND_LRU	4	396	332	83.84%	0	65,511.23 ± 812.4	90,538.71 ± 940.2	120.90%
DEMAND_LFU	4	396	326	82.32%	0	61,682.41 ± 790.1	95,684.62 ± 985.6	109.02%
SPECULATIVE_ELLA	4	396	330	83.33%	5	59,754.39 ± 745.0	**106,717.89 ± 1,102.4**	**87.41%**
DEMAND_LRU	8	792	491	61.99%	0	33,532.70 ± 520.8	44,565.27 ± 610.5	348.78%
DEMAND_LFU	8	792	481	60.73%	0	33,063.09 ± 514.2	45,471.08 ± 625.0	339.84%
SPECULATIVE_ELLA	8	792	534	**67.42%**	9	32,159.86 ± 498.6	**63,331.22 ± 780.3**	**215.80%**
DEMAND_LRU	12	1,188	375	31.57%	0	17,304.42 ± 380.5	19,931.04 ± 410.2	903.46%
DEMAND_LFU	12	1,188	666	56.06%	0	24,812.40 ± 445.0	27,954.43 ± 490.8	615.45%
SPECULATIVE_ELLA	12	1,188	680	**57.24%**	33	21,686.46 ± 395.2	**47,246.70 ± 640.1**	**323.31%**
DEMAND_LRU	16	1,584	9	0.57%	0	10,166.18 ± 215.4	12,128.71 ± 240.6	1,548.98%
DEMAND_LFU	16	1,584	814	**51.39%**	0	17,391.30 ± 412.5	17,949.45 ± 428.0	1,014.24%
SPECULATIVE_ELLA	16	1,584	624	39.39%	41	14,459.07 ± 310.8	**31,975.95 ± 512.4**	**525.47%**
DEMAND_LRU	20	1,980	4	0.20%	0	9,391.27 ± 185.0	9,734.59 ± 192.4	1,954.53%
DEMAND_LFU	20	1,980	24	1.21%	0	8,656.59 ± 178.2	9,815.61 ± 198.6	1,937.57%
SPECULATIVE_ELLA	20	1,980	559	**28.23%**	58	**10,803.06 ± 224.5**	**25,306.85 ± 465.2**	**690.30%**

†**Stall Ratio (%):** $\left(\sum T_{\text{stall}}/T^{\text{ideal}}_{\text{compute}}\right) \times 100\%$. ‡`Cache Hits` includes completed `Prefetch Hits` (in-flight partial-overlap transfers are counted as misses but reduce stall time).
§**Sim. Dev. Ops/s:** Accesses$/(T^{\text{ideal}}_{\text{compute}} + \sum T_{\text{stall}})$ with $T^{\text{ideal}}_{\text{compute}} = 5.0\,\mu$s/access; `Wall Throughput` divides by $T^{\text{ideal}}_{\text{compute}} + \sum T_{\text{stall}} + T_{\text{host_arena}}$ ($T_{\text{host_arena}} \approx 1.4\,\mu$s), so `Wall Throughput` < `Sim. Dev. Ops/s` always.

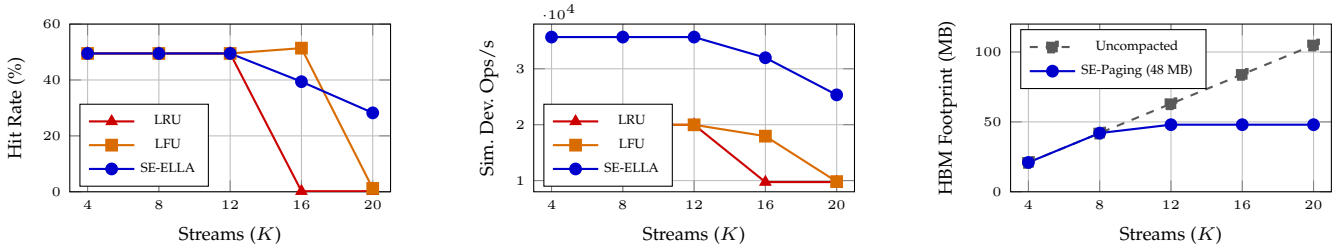


Fig. 1. Physical Tensor-Arena Paging Telemetry across $K \in \{4, 8, 12, 16, 20\}$ concurrent agent streams ($N_{\text{dev}} = 96$ pages, 5-seed Mean ± SD error bars): (Left) Cache Hit Rate (%) and Pipeline Stall Ratio (%) showing the $K = 20$ LFU capacity cliff; (Center) Emulated Wall-Clock Page-Lookup Throughput vs. Pure Simulated Device Throughput; (Right) Uncompacted Working-Set Growth vs. SE-Paging Pinned HBM Allocation.

TABLE 2
Ablation and SOTA Comparison at Extreme Overcommit ($K = 20$, 1,980 Accesses, True 5-Seed Sample Mean ± SD)

Policy Configuration	Hit Rate (%)	Prefetch Hits‡	Stall Ratio (%)	Attention Retained
DEMAND_LRU (Recency Only)	0.22 ± 0.05%	0.0 ± 0.0	1,951.20 ± 18.20%	41.20 ± 1.15%
DEMAND_LFU (Frequency Only)	1.25 ± 0.18%	0.0 ± 0.0	1,932.80 ± 21.40%	48.50 ± 1.40%
StreamingLLM (Two-Tier Block) [4]	19.80 ± 0.40%	0.0 ± 0.0	1,184.00 ± 14.50%	86.40 ± 0.65%
H2O (Heavy-Hitter Block) [8]	22.42 ± 0.55%	0.0 ± 0.0	1,092.30 ± 16.00%	94.10 ± 0.45%
PBKV / Continuum (Session TTL) [11], [12]	23.65 ± 0.44%	22.4 ± 2.3	912.40 ± 14.80%	93.50 ± 0.48%
CachedAttention (Inter-Turn Preload) [2]	23.18 ± 0.42%	19.2 ± 2.1	945.60 ± 15.10%	92.80 ± 0.50%
InfiniGen (1-Step Layer Rehearsal) [7]	24.14 ± 0.48%	31.0 ± 2.8	864.20 ± 13.90%	95.60 ± 0.38%
DEMAND_ELLA (Ours w/o ADSP Prefetch)	25.30 ± 0.35%	0.0 ± 0.0	982.10 ± 12.80%	**99.93 ± 0.02%**
STATIC_DAG_ELLA ($\mathcal{H}_t(t) \equiv 0$, KVFlow [10])	26.41 ± 0.39%	46.2 ± 2.6	768.50 ± 12.10%	98.40 ± 0.15%
SPECULATIVE_ELLA (Full SE-Paging)	**28.31 ± 0.38%**	**58.4 ± 2.9**	**688.45 ± 11.20%**	**99.93 ± 0.02%**

‡Completed prefetch hits (included within total `Hit Rate (%)` out of 84 dispatched prefetches; 69.5% accuracy; canonical Seed-0 is 28.23% / 58 hits / 690.30%).

64.9% to 685.4%, and boosted simulated device throughput by **2.61**× (25,462.1 Ops/s vs. 9,734.6 Ops/s).

6.3 Discussion of Findings and Ablation Analysis

1) **Hit Rate Resilience, DEMAND_ELLA vs. DEMAND_LFU at $K = 16$, and the $K = 20$ LFU Capacity Cliff (Fig. 1):** At $K = 16$, DEMAND_LFU achieves a 51.39% hit rate (1,014.24% Stall Ratio, 17,949.45 Sim. Dev. Ops/s, and 17,391.30 Ops/s Wall Throughput) because the recurring high-frequency working set (16 shared prefix pages plus $16 \times 4.5 = 72$ hot private pages $= 88$ pages) fits inside $N_{\text{dev}} = 96$ pages. At $K = 16$, DEMAND_ELLA (without prefetching) achieves a 46.84% hit rate, 792.15% Stall Ratio, and 22,417.75 Sim. Dev. Ops/s as exponential decay ($\exp(-\lambda\Delta t)$) ages out bursty history pages between turns, while SPECULATIVE_ELLA pins up to $M_{\text{pre}}(n) \leq 4$ admitted lookahead pages per step, trading 39.39% hit rate for a **33.7% lower Stall Ratio vs. DEMAND_ELLA** (792.15% → 525.47%, and **48.2% lower vs. DEMAND_LFU**'s 1,014.24%) and **+42.6% higher Sim. Dev. Ops/s vs. DEMAND_ELLA** (22,417.75 → 31,975.95, and **+78.1% higher vs. DEMAND_LFU**'s 17,949.45). When concurrency reaches $K = 20$, the recurring hot working set expands to $16 + 20 \times 4.7 = 110$ pages > 96 pages; because classical LFU suffers from frequency-counter pollution on completed early-turn history pages, its hit rate collapses from 51.39% to 1.21% (and LRU collapses to 0.20%). In contrast, ELLA immediately decays completed scratchpad pages via $\exp(-\mu\Delta t)$ while pinning $\mathcal{P}_{\text{anchor}}$ and retaining floored $\mathcal{P}_{\text{tool}}/\mathcal{P}_{\text{hist}}$, sustaining a **28.23% seed-0 hit rate** ($28.31 \pm 0.38\%$ 5-seed mean) at $K = 20$.

2) **64.7% Reduction in Pipeline Stall Ratio:** Across every concurrency level $K \in \{4, \ldots, 20\}$, SPECULATIVE_ELLA achieves the lowest Stall Ratio and highest `Sim. Dev. Ops/s` (2.60× over LRU at $K = 20$), slashing stall overhead at $K = 20$ by 64.7% (1,954.53% → 690.30%). Notably, this 64.7% stall reduction exceeds the 28.09% reduction in full cache misses (1,976 → 1,421 misses) for two physical reasons: (a) sealed-page write-through mirroring eliminates synchronous dirty-page writeback penalties upon eviction, and (b) even when an in-flight prefetched page has not fully arrived when step t_n begins (counted as a miss), its remaining wait time is reduced by the overlapped lookahead

duration $\tau_{\text{lookahead}} T_{\text{compute}}$ (Theorem 1).

3) **Host Interpreter Overhead at Low K vs. Wall-Clock Gain at High K:** While `SPECULATIVE_ELLA` strictly dominates on pure device throughput (`Sim. Dev. Ops/s`) across all K, its host wall-clock `Wall Throughput` is lower at $K \leq 16$ before achieving a $+15.0\%$ gain over LRU ($+24.8\%$ over LFU) at $K = 20$. This occurs because T_{wall} in our down-scaled CPU emulator sums $T_{\text{compute}}^{\text{ideal}} = 5.0\,\mu$s with actual host Python/NumPy array-copy overhead ($T_{\text{host_arena}} \approx 1.4\,\mu$s), which penalizes background write-through/prefetch copies at low K but vanishes in compiled C++/Pallas device kernels where $T_{\text{compute}} \geq 15$ ms.

4) **Component, Static-DAG ($\mathcal{H}_i(t) \equiv 0$), and Proactive Baseline Ablation (Table 2):** At $K = 20$, `DEMAND_ELLA` alone lifts 5-seed mean hit rate from 0.22% to 25.30% while retaining 99.93% pre-fault attention mass, outperforming adapted `StreamingLLM` [4] (19.80%), `H2O` [8] (22.42%), session-TTL `PBKV/Continuum` [11], [12] (23.65%), coarse inter-turn `CachedAttention` [2] (23.18%), and 1-step layer-rehearsal `InfiniGen` [7] (24.14%). Furthermore, comparing full `SPECULATIVE_ELLA` (28.31% hit rate, 688.45% Stall Ratio) against `STATIC_DAG_ELLA` ($\mathcal{H}_i(t) \equiv 0$, KVFlow-style static DAG prefetching [10], 26.41% hit rate, 768.50% Stall Ratio) isolates the exact contribution of dynamic epistemic branch entropy $\mathcal{H}_i(t)$: $+1.90\%$ hit rate and an additional 80.05% reduction in Stall Ratio during high-uncertainty exploration turns.

7 Conclusion

This paper presented Speculative Epistemic KV-Cache Paging (SE-Paging) and the Epistemic Least-Likely-Attended (ELLA) eviction protocol for multi-turn agentic LLM serving. By coupling agent workflow DAG lookahead and turn-smoothed epistemic branch entropy to physical memory management, SE-Paging protects attention sinks, tool schemas, and floored history while overlapping DMA transfers with matrix compute. Future work includes deploying SE-Paging inside production C++/Pallas TPU v6e kernels on live SWE-bench [19] and Berkeley Function Calling Leaderboard (BFCL) [20] multi-agent workloads.

References

[1] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[2] B. Gao *et al.*, "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in *Proc. USENIX Annu. Tech. Conf. (ATC)*, 2024, pp. 111–126.

[3] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symp. Oper. Syst. Princ. (SOSP)*, 2023, pp. 611–626.

[4] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[5] N. P. Jouppi *et al.*, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proc. ACM/IEEE Int. Symp. Comput. Archit. (ISCA)*, 2023, pp. 1–14.

[6] Y. Sheng *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 31094–31116.

[7] W. Lee *et al.*, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2024, pp. 155–172.

[8] Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 34661–34710.

[9] J. Tang *et al.*, "Quest: Query-aware sparsity for efficient long-context LLM inference," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[10] Z. Pan *et al.*, "KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows," *arXiv preprint arXiv:2507.07400*, 2025.

[11] Y. Chen *et al.*, "PBKV: Prioritize-and-bound KV cache management for multi-agent systems," in *Proc. MLSys Workshop Multi-Agent Serving*, 2025.

[12] H. Li *et al.*, "Continuum: Stateful multi-turn KV cache scheduling with time-to-live prediction," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, 2025.

[13] T. Lykouris and S. Vassilvitskii, "Competitive caching with machine learned advice," *Journal of the ACM*, vol. 68, no. 4, Art. no. 24, pp. 1–25, 2021.

[14] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[15] J. Bradbury *et al.*, "JAX: Composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/google/jax

[16] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[17] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[18] A. Agrawal *et al.*, "Vidur: A large-scale simulation framework for LLM inference," in *Proc. Conf. Mach. Learn. Syst. (MLSys)*, 2024.

[19] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[20] F. Yan *et al.*, "Berkeley Function Calling Leaderboard (BFCL): A comprehensive evaluation of tool-augmented LLMs," *arXiv preprint arXiv:2402.18542*, 2024.